

python - numpy and pandas

title: "NumPy, Pandas & Matplotlib - Quiz Prep Notes" tags

- python
- numpy
- pandas
- matplotlib
- data-analysis
- quiz-prep source: "data-processing.ipynb"

NumPy, Pandas & Matplotlib — Quiz Prep Notes

These notes walk through **every single cell** of your `data-processing.ipynb` notebook, in order. The notebook itself is a small real-world data cleaning project on the **Titanic dataset**. Think of it as a story: load data → look at it → clean it → engineer new columns → validate it → convert pieces of it into NumPy arrays for math.

I am writing this assuming you know core Python (lists, loops, functions) but are **new to NumPy** and only lightly familiar with Pandas. So every new concept gets a plain-English definition before we look at the code.

🔥 **How to use these notes** Each section = one or more notebook cells. Read the **Concept** box first, then the **Code**, then the **Line-by-line** explanation. The **Why it matters** line is usually what gets asked in quizzes.

0. The Big Picture — What Are These 3 Libraries?

Before anything else, you need to know *why* these three libraries always show up together.

Library	What it is	Analogy
NumPy	A library for fast numerical computing using arrays (grids of numbers)	The engine — does the heavy math
Pandas	A library for working with tabular data (rows & columns, like Excel)	The spreadsheet — built ON TOP of NumPy
Matplotlib	A library for plotting/visualizing data	The artist — draws charts from your data

🔑 **Key relationship to remember** A Pandas **DataFrame** (table) is basically a collection of Pandas **Series** (columns), and internally, a Series stores its data in a **NumPy array**. That's why you can convert a DataFrame column into a NumPy array so easily — it's already "NumPy under the hood."

1. Importing the Libraries

Code (Cell 0)

```
1 # importing necessary libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
```

Explanation

- `import numpy as np` → loads NumPy and gives it the nickname `np`. This is a **universal convention** — every NumPy user in the world writes `np`, not `numpy`. Quiz-safe fact.
- `import pandas as pd` → same idea, Pandas is always aliased as `pd`.
- `import matplotlib.pyplot as plt` → Matplotlib is a big library; `pyplot` is the specific sub-module used for plotting (like a "drawing toolkit" inside it), aliased as `plt`.

Why it matters

Without these aliases, you'd have to write `numpy.array()` every time instead of `np.array()`. Aliases save typing and are a strict industry convention — using a different alias (like `import numpy as n`) works, but nobody does it, so quizzes expect `np`, `pd`, `plt`.

2. Loading a Dataset from the Internet

Code (Cell 1)

```
1 url = "https://raw.githubusercontent.com/mwaskom/seaborn-
  data/refs/heads/master/titanic.csv"
2 df_raw = pd.read_csv(url)
```

Concept: `pd.read_csv()`

This is Pandas' most-used function. **CSV** = "Comma-Separated Values," a plain-text way to store tables (each line is a row, values separated by commas). `pd.read_csv()` reads a CSV file — whether it's on your computer OR at a web URL — and converts it into a **DataFrame**.

Concept: DataFrame

A **DataFrame** is Pandas' core object — think of it as an Excel sheet or SQL table living inside Python: rows and labeled columns, where every column can have its own data type.

Line-by-line

- `url = "..."` → just a string holding the web address of the raw CSV file.
- `df_raw = pd.read_csv(url)` → downloads/reads that CSV and stores the resulting table in the variable `df_raw`.

Why it matters

`pd.read_csv()` is the standard entry point for almost all Pandas work. It can also read local files: `pd.read_csv("myfile.csv")`.

3. Making a Safe Copy of the Data

Code (Cell 2)

```
1 df = df_raw.copy()
```

Concept: `.copy()`

In Python (and Pandas), if you just wrote `df = df_raw`, both `df` and `df_raw` would point to the **same object in memory**. Changing `df` would silently also change `df_raw`. `.copy()` creates a genuinely **independent duplicate**.

Why it matters

This is a very common quiz trap: *"What's the difference between `df = df_raw` and `df = df_raw.copy()`?"* Answer: the first is a reference (alias), the second is a true copy. The

notebook keeps `df_raw` untouched as the "original" so it can be compared later (see Cell 32) against the cleaned `df`.

4. Peeking at the Data — `head()` and `tail()`

Code (Cells 3 & 4)

```
1 df.head(10) # first 10 rows
2 df.tail(7)  # last 7 rows
```

Concept

- `.head(n)` → shows the **first** `n` **rows** of the DataFrame. Default is 5 if you don't pass a number.
- `.tail(n)` → shows the **last** `n` **rows**.

Why it matters

You almost never print an entire dataset (could be millions of rows). `.head()` and `.tail()` are the go-to "sanity check" tools to confirm data loaded correctly and see what the columns look like.

5. Checking the Shape of the Data

Code (Cell 5)

```
1 df.shape # Shows the number of rows and columns in the dataset
```

Concept: `.shape`

`.shape` is an **attribute**, not a function — notice there are no parentheses `()`. It returns a **tuple**: `(number_of_rows, number_of_columns)`.

 **Attribute vs Method** `.shape` has no `()` because it's a stored property. `.head()` has `()` because it's a method (a function that does work when called). This distinction shows up often in quizzes — `.shape` is NumPy-flavored terminology too, since NumPy arrays also have a `.shape` attribute describing their dimensions.

Why it matters

`.shape` is the fastest way to know "how big is my dataset?" — e.g., `(891, 15)` means 891 rows and 15 columns.

6. Listing Column Names

Code (Cell 6)

```
1 print(list(df.columns))
```

Concept: `.columns`

Another attribute (no parentheses). It returns an **Index object** containing all column names. Wrapping it in `list(...)` converts it into a plain Python list for cleaner printing.

Why it matters

Useful when you have many columns and want to know their exact names (spelling matters — `df['Fare']` and `df['fare']` are different in Pandas, which is case-sensitive).

7. Checking Data Types of Each Column

Code (Cell 7)

```
1 print(df.dtypes) # Displays the data types of each column in the dataset
```

Concept: `.dtypes`

Every column in a DataFrame has a **data type** (dtype), similar to how variables have types in any language. Common Pandas dtypes:

dtype	Meaning
int64	whole numbers
float64	decimal numbers (also used when a numeric column has missing values!)
object	text/strings, or mixed types
bool	True/False
category	a fixed, limited set of labels (we'll see this later)
datetime64	dates/times

Why it matters

Wrong dtypes cause bugs (e.g., trying to do math on a text column). This is also your first hint at *why* a numeric column like `age` might become `float64` instead of `int64` — a missing value (`NaN`) forces the whole column to be treated as decimal.

8. Getting a Full Summary — `.info()`

Code (Cell 8)

```
1 df.info() # Provides a concise summary of the DataFrame, including the
            number of non-null entries and memory usage
```

Concept

`.info()` is a **method** that prints a compact report combining several things at once:

- Total number of rows and columns
- Column names + how many **non-null** (i.e., non-missing) values each has
- The dtype of each column
- Total memory usage

Why it matters

This is usually the **very first command** run after loading any dataset — it tells you immediately which columns have missing data (compare "non-null count" to total row count) and whether dtypes look correct.

9. Descriptive Statistics — `.describe()`

Code (Cells 9 & 10)

```
1 df.describe()           # summary stats for numeric columns only
2 df.describe(include='all') # summary stats for ALL columns (numeric +
                             categorical)
```

Concept

`.describe()` computes summary statistics automatically. For **numeric** columns you get:

Stat	Meaning
count	number of non-missing values
mean	average
std	standard deviation (spread/variability)
min	smallest value
25%	1st quartile (25% of values fall below this)
50%	median (2nd quartile)
75%	3rd quartile
max	largest value

For **categorical/text** columns (when you pass `include='all'`), you instead get:

Stat	Meaning
unique	number of distinct values
top	most frequent value
freq	how many times that top value appears

Why it matters

This single line gives you a fast statistical fingerprint of your entire dataset — great for spotting outliers (e.g., an unrealistic `max` age) or imbalance (e.g., one `embark_town` dominating).

10. Visualizing Distribution — Matplotlib Histogram

Code (Cell 11)

```
1 plt.figure(figsize=(10, 6))
2 plt.hist(df['fare'], bins=30, edgecolor='black')
3 plt.xlabel('Fare')
4 plt.ylabel('Frequency')
5 plt.title('Distribution of Fare')
6 plt.show()
```

Concept: Histogram

A **histogram** shows how values in a numeric column are *distributed* — it groups values into ranges ("bins") and shows how many data points fall into each range as bars.

Line-by-line (this is the standard Matplotlib "recipe" — memorize this pattern)

- `plt.figure(figsize=(10, 6))` → creates a new blank figure/canvas, `figsize=(width, height)` in inches.
- `plt.hist(df['fare'], bins=30, edgecolor='black')` → the actual plotting call.
 - `df['fare']` → the data being plotted (the whole `fare` column).
 - `bins=30` → split the fare range into 30 equal-width buckets.
 - `edgecolor='black'` → draws a black outline around each bar so they're visually separated.
- `plt.xlabel(...)` / `plt.ylabel(...)` → label the horizontal and vertical axes.
- `plt.title(...)` → adds a title above the chart.
- `plt.show()` → renders/displays the chart. Without this, in some environments the plot may not appear.

Why it matters

This is the canonical Matplotlib workflow: **create figure** → **plot** → **label** → **show**. Every Matplotlib chart type (`plt.hist` , `plt.bar` , `plt.scatter` , `plt.plot`) follows this same 5-step shape.

11. Central Tendency — Mean vs Median

Code (Cell 12)

```
1 print('Mean fare:', df['fare'].mean())
2 print('Median fare:', df['fare'].median())
```

Concept

- **Mean** = the arithmetic average (sum of all values ÷ count of values).
- **Median** = the *middle* value when data is sorted — the value at the 50th percentile.

Why it matters (classic quiz question)

Mean is **sensitive to outliers**; median is **not**. Ticket fares include a few extremely expensive first-class tickets, which pull the mean upward — that's usually why `mean > median` for `fare` in this dataset. If someone asks "why use median instead of mean here?" — the answer is: *skewed data with outliers*.

12. Exploring Categorical Columns — `value_counts()`

Code (Cell 13)

```
1 print(df['embark_town'].value_counts(normalize=True))
```

Concept: `value_counts()`

Counts how many times **each unique value** appears in a column — like a frequency table.

- `normalize=True` → instead of raw counts, shows **proportions** (0 to 1, or think of it as a percentage). This is the parameter actually used here.
- (Commented-out alternatives in the cell: plain `value_counts()` gives raw counts; `dropna=False` would also count missing/ `NaN` values instead of silently ignoring them.)

Why it matters

This is *the* tool for understanding categorical (non-numeric) columns — the equivalent of `.describe()` but for text/category data.

13. Detecting Missing Values — `isna()`

Code (Cell 14)

```
1 print(df.isna().sum()) # Shows the total number of missing values in each column
```

Concept: `isna()`

`.isna()` checks **every single cell** in the DataFrame and returns `True` if it's missing (`NaN` — "Not a Number", Pandas' way of representing "no value") and `False` otherwise. The result is a same-shaped DataFrame of `True / False` values.

Concept: chaining `.sum()`

Because Python treats `True` as `1` and `False` as `0`, calling `.sum()` on that True/False table **adds them up per column**, giving you a per-column count of missing values in one line.

Why it matters

This is the standard two-step combo (`isna()` + `sum()`) used everywhere in data cleaning to answer: *"which columns have missing data, and how much?"*

14. Visualizing Missing Values

Code (Cell 15)

```
1 missing_values = df.isna().sum()
2
3 plt.figure(figsize=(10, 6))
4 missing_values.plot(kind='bar', color='skyblue')
5 plt.title('Missing Values in Each Column')
6 plt.xlabel('Columns')
7 plt.ylabel('Number of Missing Values')
8 plt.xticks(rotation=45)
9 plt.show()
```

Concept: Pandas' built-in `.plot()`

Interesting detail: this chart is **not** made with `plt.bar(...)` directly. Instead, `missing_values` is a Pandas **Series** (one column of counts), and Pandas

Series/DataFrames have their own convenient `.plot()` method that calls Matplotlib *behind the scenes*.

- `.plot(kind='bar', color='skyblue')` → tells Pandas to draw a bar chart, colored sky-blue.
- `plt.xticks(rotation=45)` → rotates the x-axis labels (column names) by 45° so long names don't overlap.

Why it matters

Quiz-relevant distinction: **you can plot directly from Pandas objects** (`series.plot()`) OR use raw Matplotlib (`plt.hist()` , `plt.bar()`). Pandas' `.plot()` is just a convenient shortcut that still uses Matplotlib underneath — they are not two competing libraries, one is built on the other.

15. Grouping Data — `groupby()`

Code (Cell 16)

```
1 print(df.groupby(['pclass'])['age'].median())
```

Concept: `groupby()`

This is one of the most powerful Pandas tools. It follows the "**split** → **apply** → **combine**" pattern:

1. **Split**: divide the DataFrame into groups based on a column's values (here, by `pclass` — passenger class 1, 2, or 3).
2. **Apply**: run a calculation on each group separately (here, `.median()` on the `age` column).
3. **Combine**: gather the results back into a single summary.

Line-by-line

`df.groupby(['pclass'])['age'].median()` reads as: "Group the rows by `pclass`, look only at the `age` column within each group, and compute the median age per group."

Why it matters

This answers questions like *"was the median age different across 1st, 2nd, and 3rd class passengers?"* — a single line replaces what would otherwise be a manual loop with filtering.

16. Deciding How to Handle a Problematic Column

Code (Cell 17)

```
1 # What can we do about deck?
2 # Option 1: Drop the column
3 # df = df.drop(columns=['deck'])
4 df.head()
```

Concept

This cell is mostly **commentary/decision-making**, showing that the `deck` column has a *lot* of missing values (you'd have seen this from `.isna().sum()` earlier). The commented-out line shows **Option 1**: delete the column entirely with `df.drop(columns=['deck'])`.

Why it matters

This is real data-science thinking, not just code: when a column is mostly empty, you must *decide* — delete it, fill it, or extract partial information from it. This notebook chooses **not** to fully drop it (see next section) and instead keeps a signal from it.

17. Creating a "Missingness Flag" Column

Code (Cell 18)

```
1 # Option 2: Preserve whether the deck information exist or not
2 df['deck_known'] = df['deck'].notna().astype(int)
3 df.head()
```

Concept: `notna()` + `astype(int)`

- `.notna()` → the exact opposite of `.isna()`. Returns `True` if a value **is present**, `False` if missing.
- `.astype(int)` → converts those `True / False` values into `1 / 0` integers.

Line-by-line

`df['deck_known'] = df['deck'].notna().astype(int)` creates a **brand-new column** called `deck_known`, where `1` means "we know this passenger's deck" and `0` means "deck info was missing."

Why it matters

This is a clever alternative to deleting information: instead of throwing away the whole `deck` column just because it's mostly missing, you keep a simplified "was it known or not" signal — sometimes *whether data is missing* is itself useful information (e.g., only surviving/documented passengers might have deck records).

18. Grouped Median for a Different Column

Code (Cell 19)

```
1 df.groupby(['who'])['age'].median()
```

Same concept as Section 15, just grouped by the `who` column (e.g., man/woman/child) instead of `pclass`. This step is groundwork for the next cell.

19. Filling Missing Values Using Group Statistics — `transform()`

Code (Cell 20)

```
1 group_age_median = (  
2     df.groupby(['who'])['age'].transform('median')  
3 )  
4  
5 df['age'] = df['age'].fillna(group_age_median)
```

Concept: `.transform()`

This is the key new idea here, and it's a common source of confusion:

- `groupby(...).median()` (Section 18) returns **one value per group** (a short summary table).

- `groupby(...).transform('median')` returns a value **for every single original row**, but that value is *that row's group median*, broadcast back to the full original length/shape.

In other words: `transform` gives you the group's median **repeated** for each row belonging to that group, so it lines up row-for-row with the original DataFrame.

Concept: `.fillna()`

`.fillna(value)` replaces every missing (`NaN`) entry in a column with the given `value` . Here, instead of one fixed number, `value` is itself a whole column (`group_age_median`) — so each missing age gets filled with *the median age of that specific person's who group* (man/woman/child), not a single global median.

Why it matters

This is smarter missing-value imputation than "fill everything with the overall average." A child's missing age is filled using the *median age of children*, not the median age of all passengers — much more realistic. This exact `groupby().transform() + fillna()` combo is extremely common in real-world data cleaning and shows up often in interviews/quizzes.

20. Dropping Rows with Missing Values — `dropna()`

Code (Cell 21)

```
1 df = df.dropna(subset=['embarked'])
2 df = df.dropna(subset=['embarked']) # example - actually
  df.dropna(subset=['embark_town'])
```

Actual code:

```
1 df = df.dropna(subset=['embarked'])
2 df = df.dropna(subset=['embark_town'])
```

Concept: `.dropna()`

Unlike `.fillna()` (which fills gaps), `.dropna()` **removes entire rows** that have missing values.

- `subset=['embarked']` → only checks the `embarked` column for missingness — rows missing in *other* columns are left alone.

Why it matters

There's an important design decision baked into this notebook:

- **Numeric column (`age`) with lots of missing values** → filled in smartly using group medians (Section 19), because dropping those rows would lose too much data.
- **Categorical columns (`embarked` , `embark_town`) with very few missing values** → simply dropped, because losing a handful of rows barely affects the dataset, and there's no sensible "average" to fill text categories with.

This contrast (fill vs. drop, depending on how much is missing and what type of column it is) is a classic exam concept.

21. Finding and Removing Duplicate Rows

Code (Cells 22 & 23)

```
1 df.duplicated().sum()      # Check for duplicate rows and count them
2
3 df = df.drop_duplicates()  # Remove duplicate rows
4 df.reset_index(drop=True, inplace=True) # Reset the index afterward
```

Concept: `.duplicated()`

Returns `True` / `False` for each row — `True` if that row is an **exact duplicate** of a previous row (all column values identical). `.sum()` again converts True/False into a count.

Concept: `.drop_duplicates()`

Removes duplicate rows, keeping only the first occurrence of each.

Concept: `.reset_index(drop=True, inplace=True)`

Every DataFrame has a row **index** (like row numbers). After deleting rows, the index becomes "gappy" (e.g., 0, 1, 3, 4, 7, ...) because the removed rows' index numbers vanish. `.reset_index()` renumbers the index cleanly from 0 upward again.

- `drop=True` → don't keep the old messy index as a new column (which is the default behavior otherwise).
- `inplace=True` → modify `df` directly rather than returning a new DataFrame that you'd have to reassign.

Why it matters

Leftover gaps in the index aren't "wrong," but they can cause confusing bugs later (e.g., `df.loc[3]` behaving unexpectedly). Resetting the index after any row-deletion operation is standard best practice.

22. Feature Engineering — Creating a New Column

Code (Cell 24)

```
1 df['family_size'] = (  
2     df['sibsp'] + df['parch'] + 1  
3 )  
4  
5 df[['sibsp', 'parch', 'family_size']].head(10)
```

Concept: Vectorized column arithmetic

`df['sibsp'] + df['parch'] + 1` adds two entire columns together, **element-by-element, all at once** — no loop needed. `sibsp` = siblings/spouses aboard, `parch` = parents/children aboard, `+1` accounts for the passenger themselves. The result is a brand-new column, `family_size`.

Concept: Selecting multiple columns

`df[['sibsp', 'parch', 'family_size']]` — note the **double square brackets**. A single bracket `df['col']` returns one column (a Series). Double brackets with a **list** of names `df[['col1', 'col2']]` returns a mini-DataFrame with just those columns.

Why it matters

This is called **feature engineering** — building new, more useful columns out of existing raw data. It's also your first real look at **vectorization**: NumPy/Pandas let you operate on entire columns simultaneously instead of writing a Python `for` loop, which is both faster and cleaner.

23. Changing Data Types to `category`

Code (Cell 25)

```
1 df['embarked'] = df['embarked'].astype('category')
2 df['embark_town'] = df['embark_town'].astype('category')
3 df.dtypes
```

Concept: `.astype()`

A general-purpose method to convert a column's data type. Here it converts text columns into the special `category` dtype.

Concept: `category` dtype

Used when a column has a **small, fixed set of repeating values** (like `embarked` : only `S` , `C` , `Q`). Instead of storing the same strings over and over, Pandas stores them more efficiently internally (like an enum) — same displayed values, but less memory and often faster grouping operations.

Why it matters

Quiz-relevant: `category` is a memory/performance optimization for **low-cardinality** text columns (few unique values). It's *not* used for something like a `name` column, where nearly every value is unique.

24. Re-checking Shape After Cleaning

Code (Cell 26)

```
1 df.shape
```

Same concept as Section 5 — but now used to confirm how many rows remain **after** dropping duplicates and missing `embarked` / `embark_town` rows. Comparing this to the original `df_raw.shape` tells you exactly how much data was removed during cleaning.

25. Data Validation — Range Checks

Code (Cells 27 & 28)

```
1 print(df['age'].between(0, 200).all()) # Check if all age values are
   between 0 and 200
2 print((df['fare'] >= 0).all())         # Check if all fare values are
   non-negative
```

Concept: `.between()`

`.between(a, b)` checks each value in a column and returns `True / False` depending on whether it falls within the range `[a, b]` (inclusive).

Concept: Boolean comparisons on columns

`df['fare'] >= 0` compares **every value** in the column to `0` at once, producing a full column of `True / False`.

Concept: `.all()`

`.all()` checks whether **every single value** in a True/False Series is `True`. If even one value is `False`, `.all()` returns `False`.

Why it matters

This is **data validation / sanity checking** — confirming your cleaned data makes logical sense (no negative fares, no impossible ages) before you trust it for analysis. `.all()` pairs with boolean conditions constantly in both Pandas and NumPy.

26. Data Validation — `assert with isin()`

Code (Cell 29)

```
1 assert df['survived'].isin([0, 1]).all()
```

Concept: `.isin()`

Checks each value in a column against a **list of allowed values**, returning `True / False` per row — here, checking that every value in `survived` is either `0` or `1` (and nothing else, like a typo `2` or a missing value).

Concept: `assert`

A core Python keyword (not Pandas-specific!). `assert condition` does nothing if `condition` is `True`. If `condition` is `False`, it immediately **stops the program** and raises an `AssertionError`.

Why it matters

Combining `assert` with a validation check like this is a common pattern for **enforcing data quality**: "if this assumption about my data is ever violated, crash loudly right away instead of silently producing wrong results later."

27. NumPy Fundamentals (Needed Before the Next Cells)

Before looking at Cells 30–32, let's properly build up NumPy since this is the part you're least familiar with.

What is a NumPy array?

A NumPy **array** (technically an `ndarray`, "n-dimensional array") is a grid of values, **all of the same data type**, that NumPy can process extremely fast.

```
1 import numpy as np
2 a = np.array([1, 2, 3, 4])
```

Why not just use a Python list?

Python list	NumPy array
Can hold mixed types (<code>[1, "a", 3.5]</code>)	Must hold one consistent type
Slow for math (needs manual loops)	Extremely fast (written in optimized C code internally)
<code>[1,2,3] + [1,2,3] → [1,2,3,1,2,3]</code> (concatenation!)	<code>np.array([1,2,3]) + np.array([1,2,3]) → [2,4,6]</code> (element-wise math)

This last row is the single most important idea in NumPy: operations happen **element-by-element automatically** — this is called **vectorization**, and it's why NumPy (and Pandas, built on it) is so much faster than plain Python loops for numeric work.

How does this connect to Pandas?

A Pandas column (Series) is, underneath, stored as a NumPy array. `.to_numpy()` (used next) simply "unwraps" a Pandas Series and hands you the raw NumPy array directly — stripping away the row-index labels and just giving you pure numbers.

28. Converting Pandas Columns into NumPy Arrays

Code (Cell 30)

```
1 age_array = df['age'].to_numpy()
2 fare_array = df['fare'].to_numpy()
```

Concept: `.to_numpy()`

Converts a Pandas Series into a plain NumPy array (dropping the Pandas-specific index labels, keeping just the raw values).

Why it matters

Sometimes you want to use NumPy's math functions directly (as in the next section), or feed data into another library (like scikit-learn) that expects raw NumPy arrays rather than Pandas objects. This is the standard "bridge" between the two libraries.

29. NumPy Statistical Functions — `max` and `percentile`

Code (Cell 31)

```
1 print('Maximum fare: ', np.max(fare_array))
2 print('95th percentile: ', np.percentile(fare_array, 95))
```

Concept: `np.max()`

Returns the single largest value in an array. (Note: `df['fare'].max()` — the Pandas method — would give the same answer; NumPy just offers the function-style version, `np.max(array)`, which is what you use once your data is already a plain NumPy array.)

Concept: `np.percentile(array, q)`

Returns the value **below which** `q % of the data falls`. `np.percentile(fare_array, 95)` means: *"find the fare value such that 95% of all passengers paid less than or equal to this."*

- The 50th percentile = the median.
- Percentiles are useful for understanding the "shape" of data beyond a single average, especially for spotting how extreme the top values are (e.g., how much more expensive the priciest tickets were compared to typical ones).

Why it matters

`np.percentile()` is a distinctly NumPy tool (Pandas has `.quantile()` for the same idea, just using a 0–1 scale instead of 0–100). Knowing both names — **percentile (NumPy, 0–100 scale)** vs **quantile (Pandas, 0–1 scale)** — is a common quiz gotcha.

30. NumPy and Missing Values — `mean` vs `nanmean`

Code (Cell 32)

```
1 raw_age = df_raw['age'].to_numpy()
2 print(np.mean(raw_age))
3 print(np.nanmean(raw_age))
```

Concept: The problem

Recall `df_raw` is the **original, uncleaned** dataset (Section 3) — its `age` column still has missing values (`NaN`). This cell demonstrates what happens when NumPy math functions meet missing data.

Concept: `np.mean()` vs `np.nanmean()`

- `np.mean(array)` — computes the average normally. **But** if even one value in the array is `NaN`, the *entire result becomes* `NaN` ("NaN poisons the calculation" — any arithmetic involving `NaN` produces `NaN`).
- `np.nanmean(array)` — computes the average while **automatically ignoring** any `NaN` values, as if they were never there.

Why it matters (very common quiz question!)

This cell is a deliberate demonstration: running it shows `np.mean(raw_age)` printing `nan`, while `np.nanmean(raw_age)` prints a real number. This proves *why* you can't just carelessly

call `.mean()` on raw, uncleaned data — and it explains *why* the notebook bothered to properly handle missing `age` values earlier (Section 19) using `groupby().transform('median') + fillna()`, rather than leaving them as `NaN` for later analysis. NumPy has "nan-safe" versions of many functions: `np.nansum()`, `np.nanmedian()`, `np.nanstd()`, `np.nanmax()`, etc. — same idea, they just skip missing values.

Full Concept Cheat Sheet (Quick Revision)

Pandas — Inspecting Data

Command	Purpose
<code>pd.read_csv(path_or_url)</code>	Load a CSV into a DataFrame
<code>df.copy()</code>	Make an independent duplicate
<code>df.head(n)</code> / <code>df.tail(n)</code>	Preview first/last n rows
<code>df.shape</code>	(rows, columns) — attribute, no <code>()</code>
<code>df.columns</code>	List of column names — attribute
<code>df.dtypes</code>	Data type of each column
<code>df.info()</code>	Full structural summary
<code>df.describe()</code>	Numeric summary stats
<code>df.describe(include='all')</code>	Summary stats for all columns

Pandas — Cleaning Data

Command	Purpose
<code>df.isna()</code> / <code>df.notna()</code>	True/False for missing / present values
<code>df.isna().sum()</code>	Count missing values per column
<code>df.fillna(value)</code>	Replace missing values
<code>df.dropna(subset=[...])</code>	Remove rows missing values in given columns
<code>df.duplicated()</code>	Flag duplicate rows
<code>df.drop_duplicates()</code>	Remove duplicate rows
<code>df.reset_index(drop=True, inplace=True)</code>	Renumber the row index
<code>df.astype(type)</code>	Convert a column's data type

Command	Purpose
<code>df.groupby(col)[other_col].median()</code>	One summary value per group
<code>df.groupby(col)[other_col].transform('median')</code>	Group value repeated for every original row
<code>df['col'].value_counts(normalize=True)</code>	Frequency table (as proportions)
<code>df['col'].mean() / .median()</code>	Central tendency
<code>df['col'].between(a, b)</code>	Range check per value
<code>series.isin([...])</code>	Membership check per value
<code>series.all()</code>	True only if every value is True

NumPy

Command	Purpose
<code>np.array([...])</code>	Create an ndarray
<code>series.to_numpy()</code>	Convert a Pandas Series → NumPy array
<code>np.max(arr) / np.min(arr)</code>	Largest / smallest value
<code>np.mean(arr)</code>	Average (NaN-sensitive — poisons result)
<code>np.nanmean(arr)</code>	Average, ignoring NaNs
<code>np.percentile(arr, q)</code>	Value at the qth percentile (0–100 scale)

Matplotlib

Command	Purpose
<code>plt.figure(figsize=(w, h))</code>	Start a new blank chart canvas
<code>plt.hist(data, bins=n, edgecolor='black')</code>	Draw a histogram
<code>series.plot(kind='bar')</code>	Pandas shortcut that draws via Matplotlib
<code>plt.xlabel() / plt.ylabel() / plt.title()</code>	Label the chart
<code>plt.xticks(rotation=45)</code>	Rotate x-axis tick labels
<code>plt.show()</code>	Render the chart

Big Ideas to Remember for the Quiz

1. **NumPy array vs Python list:** arrays are same-type, fixed, and support fast **vectorized** (element-wise) math; lists don't.
2. NaN **"poisons" calculations:** `np.mean()` on data with missing values returns `NaN`; use `np.nanmean()` (or clean the data first) to avoid this.
3. **Attribute vs Method:** `.shape`, `.columns`, `.dtypes` have no `()`; `.head()`, `.describe()`, `.mean()` do.
4. `groupby().median()` **vs** `groupby().transform('median')`: the first collapses to one row per group; the second keeps the original row count, broadcasting the group stat back to every row — this is exactly what makes group-based `fillna()` possible.
5. **Fill vs Drop for missing data:** fill (impute) when losing rows would waste too much data and a sensible substitute value exists (e.g., group median); drop when missingness is rare and there's no reasonable value to substitute (e.g., a missing category/town).
6. **Pandas is built on NumPy:** `.to_numpy()` reveals the raw array underneath a Series; this is why Pandas and NumPy functions so often produce identical results (`df['fare'].max() ≈ np.max(fare_array)`).
7. **Percentile (NumPy, 0–100) vs Quantile (Pandas, 0–1):** same underlying concept, different scale convention.